

Projet : Application Full-Stack de Suivi de Candidatures, adapté pour ordinateur & smartphone

Nom de l'application : History.job

Stack : React.js (Front-end), Node.js / Express (Back-end), NoSQL (Base de données).

Spécificité : L'application est entièrement en anglais.

1. Objectifs du Projet.....	2
2. Périmètre Fonctionnel (Fonctionnalités).....	2
2.1. Authentification & Sécurité.....	2
2.2. Gestion des Profils "Pays" (Contextes de recherche).....	2
2.3. Gestion des Candidatures (Core Feature).....	2
2.4. Suivi et Historique des Échanges.....	3
2.5. Gestion Documentaire.....	3
3. Fonctionnalités Complémentaires (À développer plus tard).....	3
4. Contraintes Techniques & Architecture.....	4
5. Architecture Globale du Projet.....	5
5.1. Architecture globale.....	5
5.2. Backend (Node.js/Express).....	5
5.3. Frontend (React).....	5
6. Base de Données.....	6
6.1. Modèle User.....	6
6.2. Modèle Profile.....	6
6.3. Modèle Application (candidature).....	7
7. Tableau de Routes API (Endpoints).....	8
7.1. Authentification (Users).....	8
7.2. Gestion des Profils (Profiles).....	8
7.3. Gestion des Candidatures (Applications).....	9
7.4. Historiques et Contacts (Sous-documents).....	9
7.5. Gestion Documentaire (Fichiers).....	10
8. Les Vues.....	11
8.1. L'Écran de connexion / Inscription (Auth view).....	11
8.2. Le hub de sélection des profils (Profile selector).....	11
8.3. Tableau de bord principal (Main dashboard).....	11
8.4. La modale de création (New application form).....	12
8.5. La fiche détaillée de la candidature (Application Detail View).....	12
9. CI/CD et Architecture cloud.....	13
9.1. Environnement Local (Docker).....	13
9.2. Dépôt centralisé (Github).....	13
9.3. Déploiement continu (CI/CD avec Netlify & Render).....	13
9.4. La base de données Cloud (MongoDB Atlas).....	13
10. Palette graphique.....	14
Annexe — Glossaire.....	15

1. Objectifs du Projet

Conception d'une application web sécurisée permettant à un candidat de centraliser, d'organiser et de suivre l'évolution de ses recherches d'emploi à l'international. L'outil doit remplacer l'usage des tableurs traditionnels en offrant une interface visuelle, des rappels temporels et un stockage documentaire.

2. Périmètre Fonctionnel (Fonctionnalités)

2.1. Authentification & Sécurité

- **Création de compte** : Inscription avec username et mot de passe.
- **Sécurité** : Hachage des mots de passe en base de données (ex: bcrypt).
- **Connexion** : Authentification par token (ex: JWT - JSON Web Token) pour maintenir la session active en toute sécurité.

2.2. Gestion des Profils "Pays" (Contextes de recherche)

L'utilisateur peut cibler plusieurs marchés. Il doit pouvoir séparer ses recherches.

- **Création de profil** : Nom du profil, sélection d'un pays (ex : "Nouvelle Zélande") grâce à une librairie NPM locale (npm install countries-and-timezones)
- **Configuration du fuseau horaire** : Automatique grâce à la sélection d'un pays
- **Affichage de l'heure locale** : Sur le tableau de bord du profil, une horloge dynamique affiche l'heure exacte du pays ciblé, utile pour la planification des appels internationaux.

2.3. Gestion des Candidatures (Core Feature)

Au sein d'un profil Pays, l'utilisateur peut ajouter une fiche "Candidature". Champs requis pour une fiche entreprise :

- **Entreprise** : Nom de l'entreprise.
- **Détails du poste** : Intitulé du poste, Lieu, Salaire proposé, type (Ville / Hybride / Remote).
- **Contacts** : Nom du/des recruteurs, Email, Numéro de téléphone.
- **Source** : Lien URL de l'offre d'emploi (Seek, LinkedIn, etc.).
- **Statut global** : État actuel de la candidature (ex: À postuler, Envoyé, Entretien, Refusé, Offre reçue).

2.4. Suivi et Historique des Échanges

La fiche candidature doit agir comme un journal de bord temporel.

- **Date de candidature** : Sélection de la date d'envoi initial.
- **Historique des relances** : Possibilité d'ajouter plusieurs dates de relance avec un champ note court, en spécifiant le canal de communication (Mail, Téléphone, LinkedIn, Plateforme).
- **Historique des Réponses** : Possibilité d'ajouter plusieurs dates de réponses reçues, en spécifiant le canal de communication (Mail, Téléphone, LinkedIn, Plateforme).
- **Prise de notes** : Un champ texte libre pour préparer ses entretiens (infos sur l'entreprise, questions à poser...) où on met ce que l'on veut.

2.5. Gestion Documentaire

Pour chaque candidature, l'utilisateur doit savoir exactement ce qu'il a envoyé.

- **Upload de fichiers** : Possibilité de télécharger le CV spécifique (.pdf) et la Lettre de Motivation (.pdf/.docx) utilisés pour cette offre précise.
- **Visualisation** : Lien pour télécharger ou prévisualiser les documents depuis la fiche.

3. Fonctionnalités Complémentaires (À développer plus tard)

- **L'Alerte "Cold Track" (Rappel de relance)** : Un indicateur visuel (ex: une pastille rouge) qui s'active automatiquement sur le tableau de bord si une candidature est au statut "Envoyé" et qu'aucune réponse/relance n'a eu lieu depuis plus de 10 jours.
- **Le Dashboard Analytique** : Une petite section avec des graphiques simples (Taux de réponse, Nombre d'entretiens obtenus vs Nombre de CV envoyés) pour montrer aux recruteurs que vous savez manipuler la donnée visuelle (via une librairie comme Recharts).
- **Mode Sombre (Dark Mode)** : Un bouton toggle (toggle switch) pour basculer l'interface en mode clair/sombre.

4. Contraintes Techniques & Architecture

Environnement de Développement Global :

- **Moteur d'exécution** : Node.js v24.18.0.
- **Gestionnaire de paquets** : npm v10.x

Front-end (Dossier /frontend) :

- **Cœur de l'application** : React.js ^18.3.0 (Version très stable, compatible avec tout l'écosystème).
- **Moteur de construction** : Vite.js ^5.3.0.
- **Routage** : React Router DOM ^6.24.0
- **Style & UI** : Tailwind CSS ^3.4.0 (avec postcss ^8.4.0 et autoprefixer ^10.4.0).
- **Gestion des requêtes & Cache** : @tanstack/react-query ^5.50.0 (La v5 est la plus optimisée et se marie parfaitement avec Vite).
- **Gestion d'état global UI** : Zustand ^4.5.0.
- **Client HTTP** : Axios ^1.7.0.
- **Validation des formulaires** : Zod ^3.23.0 .
- **Data Visualisation (V2)** : Recharts ^2.12.0.

Back-end (Dossier /backend) :

- **Framework serveur** : Express.js ^4.19.0.
- **Modélisation Base de données** : Mongoose ^8.4.0 (La v8 est conçue pour Node 20+ et MongoDB Atlas).
- **Sécurité des mots de passe** : Bcrypt ^5.1.0.
- **Authentification** : Jsonwebtoken (JWT) ^9.0.0.
- **Gestion des fichiers** : Multer ^2.1.0 (Version LTS recommandée pour la sécurité).
- **Communication Cloudinary** : Cloudinary SDK Node ^2.2.0.
- **Validation des requêtes** : Zod ^3.23.0.
- **Sécurité inter-domaines** : CORS ^2.8.5.
- **Variables d'environnement** : Dotenv ^16.4.0.
- **Données géographiques** : Countries-and-timezones ^3.6.0.

Hébergement & Déploiement (CI/CD) :

- **Base de données** : MongoDB Atlas (Cluster gratuit M0).
- **Stockage de fichiers** : Cloudinary (Free Tier).
- **Front-end** : Netlify (Connecté au dossier frontend/ du dépôt GitHub).
- **Back-end** : Render (Connecté au dossier backend/ du dépôt GitHub).

Autres : l'application est en anglais, les commentaires du code, l'interface, l'affichage et le stockage des dates...

5. Architecture Globale du Projet

5.1. Architecture globale

À la racine, on aura 2 dossiers :

```
history-job/  
├── backend/ # Ton API Node.js / Express  
└── frontend/ # Ton application React.js
```

5.2. Backend (Node.js/Express)

L'architecture du back-end respecte le modèle classique et éprouvé Modèle-Contrôleur-Route (MCR), parfait pour une architecture RESTful. Chaque dossier a une responsabilité unique.

```
backend/  
├── config/ # Configuration des services externes  
│   ├── db.js # Connexion à MongoDB Atlas  
│   └── cloudinary.js # Configuration Cloudinary  
├── controllers/ # Logique métier (Ce que fait l'application)  
│   ├── authController.js # Register/login, hashage bcrypt et JWT  
│   ├── profileController.js # Création/Lecture des profils  
│   └── appController.js # Logique des candidatures et historiques  
├── middlewares/ # Les "videurs" de l'API  
│   ├── auth.js # Vérifie la validité du token JWT  
│   ├── upload.js # Configuration Multer pour les fichiers  
│   └── validate.js # Applique un schéma Zod à req.body  
├── models/ # Schémas de base de données (Mongoose)  
│   ├── User.js  
│   ├── Profile.js  
│   └── Application.js  
├── routes/ # URLs exactes de l'API  
│   ├── authRoutes.js  
│   ├── profileRoutes.js  
│   └── appRoutes.js  
├── validators/ # Schémas de validation (Zod)  
│   ├── authValidator.js # Validation username/password  
│   ├── profileValidator.js # Validation création/update profil  
│   └── appValidator.js # Validation des candidatures  
├── .env # Secrets (URL MongoDB, JWT, Cloudinary)  
├── .gitignore  
├── package.json  
└── server.js # Point d'entrée (démarré Express)
```

5.3. Frontend (React)

Séparation des "Pages" (Vues complètes) et des "Composants" (Petits boutons réutilisables) afin d'avoir un front-end moderne en React bien organisé.

```

frontend/
├── src/
│   ├── api/           # Pont de communication avec le Back-end
│   │   ├── axios.js    # Configuration Axios (injection du JWT)
│   │   ├── authApi.js
│   │   ├── profileApi.js
│   │   └── appApi.js
│   ├── assets/        # Images, icônes, index.css (Tailwind)
│   ├── components/    # Briques d'interface réutilisables
│   │   ├── common/    # Boutons, Inputs, Modales
│   │   └── layout/    # Navbar, Sidebar, Footer
│   ├── hooks/         # Gestion des requêtes serveur (TanStack Query)
│   │   ├── useProfiles.js # useQuery/useMutation pour /api/profiles
│   │   └── useApplications.js
│   ├── pages/         # Les 5 vues principales (CDC)
│   │   ├── AuthPage.jsx
│   │   ├── ProfileSelector.jsx
│   │   ├── Dashboard.jsx
│   │   └── ApplicationDetail.jsx
│   ├── schemas/       # Schémas de validation (Zod)
│   │   ├── authSchema.js
│   │   ├── profileSchema.js
│   │   └── applicationSchema.js
│   ├── store/         # Gestion d'état global UI (Zustand)
│   │   ├── authStore.js # Token JWT et infos utilisateur
│   │   └── uiStore.js   # Profil sélectionné, dark mode, modales
│   ├── utils/         # Fonctions utilitaires
│   │   ├── formatDate.js
│   │   └── timezones.js # Logique de 'countries-and-timezones'
│   ├── App.jsx        # Routes front (React Router)
│   └── main.jsx        # Point d'entrée React
├── .env               # URL de l'API back-end
├── package.json
└── tailwind.config.js # Configuration design et Dark Mode

```

6. Base de Données

6.1. Modèle User

Gère l'authentification.

```

{
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true }, // Sera hashé (crypté) avec bcrypt
  createdAt: { type: Date, default: Date.now },
  lastLogin: { type: Date, default: Date.now }
}

```

6.2. Modèle Profile

Permet de lier un utilisateur à plusieurs pays/recherches.

```

{
  userId: { type: ObjectId, ref: 'User' }, // Lien vers l'utilisateur
  profileName: { type: String, required: true }, // ex: "Développeur web"
  country: { type: String, required: true }, // ex: "Nouvelle-Zélande", "France"
}

```

```

timezone: { type: String, required: true }, // ex: "Pacific/Auckland" ou "Europe/Paris"
isActive: { type: Boolean, default: true },
createdAt: { type: Date, default: Date.now }
}

```

6.3. Modèle Application (candidature)

C'est la pièce maîtresse. Elle est rattachée à un Profil précis.

```

{
  profileId: { type: ObjectId, ref: 'Profile', required: true },

  // --- Détails de l'entreprise et du poste ---
  companyName: { type: String, required: true },
  jobTitle: { type: String, required: true },
  location: { type: String }, // Adresse
  jobType: { type: String, enum: ['C', 'H', 'R'] }, // C = City, H = Hybrid, R = Remote
  salaryExpected: { type: String },
  currency: { type: String, default: '€' },
  jobAdUrl: { type: String }, // Lien de l'offre (Seek, LinkedIn...)

  // --- Contacts (Tableau de sous-documents) ---
  contacts: [{
    name: String,
    email: String,
    phone: String,
    job: String,
  }],

  // --- Statut global ---
  status: {
    type: String,
    // Correspondance stricte : À postuler, Envoyé, Entretien, Refusé, Offre reçue
    enum: ['T', 'A', 'I', 'R', 'O'], // T = To Apply, A = Applied, I = Interviewing, R = Rejected, O =
    Offer
    default: 'T'
  },

  // --- Suivi et Historique des Échanges ---
  appliedDate: { type: Date }, // Date de candidature initiale

  // Historique des relances
  followUps: [{
    date: { type: Date, required: true },
    note: { type: String }, // Champ note court
    communicationChannel: {
      type: String,
      enum: ['M', 'P', 'L', 'I', 'S'] // M = Mail, P = Phone, L = LinkedIn, I = Intern site, S = Seek.co
    }
  }],

  // Historique des réponses
  replies: [{
    date: { type: Date, required: true },

```

```

communicationChannel: {
  type: String,
  enum: ['M', 'P', 'L', 'I', 'S'] // M = Mail, P = Phone, L = LinkedIn, I = Intern site, S = Seek.co
},
}],

// Prise de notes (Champ texte libre pour préparer les entretiens)
notes: { type: String },

// --- 2.5. Gestion Documentaire (URLs Cloudinary) ---
documents: {
  cvUrl: { type: String },
  coverLetterUrl: { type: String }
},

createdAt: { type: Date, default: Date.now },
updatedAt: { type: Date, default: Date.now }
}

```

7. Tableau de Routes API (Endpoints)

7.1. Authentification (Users)

Ces routes gèrent l'accès à l'application. C'est la seule partie de l'API qui ne nécessitera pas de Token JWT, puisqu'elles servent justement à le générer.

Méthode HTTP	Route API	Rôle exact
POST	/api/auth/register	Création de compte. Reçoit le username et le password, hache le mot de passe avec bcrypt, et sauvegarde l'utilisateur en base de données.
POST	/api/auth/login	Connexion. Vérifie les identifiants et renvoie un Token JWT au front-end pour maintenir la session active.
GET	/api/auth/me	Vérification de session. Permet au front-end React, au rafraîchissement de la page, de vérifier si le token est toujours valide et de récupérer les infos de l'utilisateur.

7.2. Gestion des Profils (Profiles)

Toutes les routes à partir d'ici sont sécurisées. Elles nécessitent que le Token JWT soit envoyé dans les en-têtes (Headers) de la requête pour identifier l'utilisateur.

Méthode HTTP	Route API	Rôle exact
POST	/api/profiles	Création. Crée un nouveau profil lié à l'ID de l'utilisateur connecté.
GET	/api/profiles	Lecture. Récupère la liste de tous les profils appartenant à l'utilisateur pour alimenter la vue "Profile selector".
GET	/api/profiles/:id	Détail. Récupère un profil spécifique et ses informations.
PUT	/api/profiles/:id	Mise à jour. Permet de renommer le profil ou de changer son statut isActive.
DELETE	/api/profiles/:id	Suppression. Supprime un profil et toutes les candidatures qui y sont rattachées.

7.3. Gestion des Candidatures (Applications)

C'est le cœur de votre application (Core Feature). Ces routes alimentent le tableau de bord principal et les fiches détaillées.

Méthode HTTP	Route API	Rôle exact
POST	/api/applications	Création. Ajoute une nouvelle fiche candidature (nom de l'entreprise, poste, lieu, etc.) rattachée à un Profil précis.
GET	/api/applications/profile/:profileId	Lecture globale. Récupère toutes les candidatures associées à un profil donné pour les afficher sur le tableau de bord (Main dashboard).
GET	/api/applications/:id	Lecture détaillée. Récupère l'intégralité d'une fiche candidature pour la page de détail.
PUT	/api/applications/:id	Mise à jour globale. Permet de modifier les informations de base (salaire, URL de l'offre), le statut (ex: de "Applied" à "Interviewing"), ou d'ajouter du texte dans la zone de notes.
DELETE	/api/applications/:id	Suppression. Supprime définitivement l'offre du tableau de bord et de la base de données.

7.4. Historiques et Contacts (Sous-documents)

Bien qu'on puisse tout faire avec la route PUT /api/applications/:id, il est plus propre et plus facile côté React de créer des routes spécifiques pour ajouter des éléments dans les tableaux (Arrays) de la base de données.

Méthode HTTP	Route API	Rôle exact
POST	/api/applications/:id/contacts	Ajoute un nouveau contact (Nom, Email, Téléphone) dans le tableau contacts de la candidature.
POST	/api/applications/:id/followups	Ajoute une nouvelle date, une note et un canal de communication dans le tableau followUps (Historique des relances).
POST	/api/applications/:id/replies	Ajoute une nouvelle réponse dans le tableau replies en spécifiant le canal (Mail, Téléphone, etc.).

7.5. Gestion Documentaire (Fichiers)

Cette route est à part car elle ne manipule pas du JSON standard, mais des données de type fichier (multipart/form-data).

Méthode HTTP	Route API	Rôle exact
POST	/api/applications/:id/upload	Upload. Réceptionne le fichier PDF (CV ou Lettre) envoyé par le front-end, l'envoie sur Cloudinary, puis met à jour la base de données avec l'URL générée.

8. Les Vues

8.1. L'Écran de connexion / Inscription (Auth view)

C'est la porte d'entrée de l'application.

- **Rôle** : Sécuriser l'accès à l'application et identifier l'utilisateur pour charger ses données.
- **Fonctionnalités** :
 - Formulaire de connexion (username + Mot de passe).
 - Formulaire de création de compte.
 - Gestion des messages d'erreur (ex: "Identifiants incorrects" ou "Cet identifiant existe déjà").

8.2. Le hub de sélection des profils (Profile selector)

Une fois connecté, l'utilisateur ne doit pas arriver directement sur ses candidatures, puisqu'il doit d'abord choisir son profil.

- **Rôle** : Permettre la navigation entre les différents pays de recherche ou en ajouter un nouveau.
- **Fonctionnalités** :
 - Affichage sous forme de grandes cartes (ex: une carte "Nouvelle-Zélande", une carte "France").
 - Affichage en temps réel de l'heure locale directement sur la carte de chaque profil.
 - Bouton "Ajouter un nouveau profil" déclenchant une modale (fenêtre superposée) pour indiquer un nom à ce profil et sélectionner un pays.

8.3. Tableau de bord principal (Main dashboard)

C'est le cœur de l'application. C'est ici que l'utilisateur atterrit après avoir cliqué sur un profil.

- **Rôle** : Offrir une vue d'ensemble instantanée de toutes les candidatures en cours pour un profil donné.
- **Fonctionnalités** :
 - Barre de navigation (Navbar) contenant le nom du profil actuel, l'horloge dynamique de la timezone, l'horloge actuelle, bouton de déconnexion, bouton de changement de profil, mode sombre ou clair (à développer plus tard).
 - Bouton d'action principal "Ajouter une candidature" très visible.
 - Vue des candidatures sous forme de tableau interactif ou de tableau Kanban (colonnes basées sur le statut : To Apply, Applied, Interviewing, etc.).
 - Petite zone analytique en haut de l'écran avec plusieurs chiffres clés (Total des candidatures, Total envoyés, Entretiens en cours, Refus...).

8.4. La modale de création (New application form)

Plutôt que de rediriger l'utilisateur vers une nouvelle page qui casse sa navigation, l'ajout d'une candidature doit se faire via une fenêtre qui s'ouvre par-dessus le tableau de bord.

- **Rôle** : Saisir rapidement les informations de base d'une offre trouvée sur Seek ou LinkedIn.
- **Fonctionnalités** :
 - Champs de saisie rapides : Nom de l'entreprise, Titre du poste, Lien de l'offre.
 - Menus déroulants pour le statut initial (par défaut sur To Apply) et le type de poste (City, Hybrid, Remote).
 - Bouton de validation qui ferme la fenêtre et rafraîchit automatiquement le tableau de bord en arrière-plan.

8.5. La fiche détaillée de la candidature (Application Detail View)

C'est la page la plus complète de l'application. On y accède en cliquant sur une ligne du tableau de bord. C'est le véritable journal de bord de l'offre ciblée.

- **Rôle** : Centraliser absolument tout le contexte d'une offre précise pour préparer ses entretiens et gérer ses documents.
- **Fonctionnalités** :
 - **En-tête (Header)** : Nom de l'entreprise, poste, et un gros menu déroulant pour changer le statut global (ex: passer de Applied à Interviewing).
 - **Section Informations** : Lieu, devise, salaire attendu, bouton cliquable redirigeant vers l'URL de l'offre originale.
 - **Section Contacts** : Bouton pour ajouter un contact (Nom, Email, Téléphone, Rôle).
 - **Section Chronologie (Timeline)** : Un bloc vertical visuel permettant d'ajouter et de visualiser rapidement la date d'envoi (appliedDate) de la candidature, les dates de relance (followUps), et les dates de réponses reçues (replies).
 - **Section Documents** : Une zone de "Glisser-Déposer" (Drag & Drop) pour importer le CV et la lettre de motivation. Une fois téléchargés (via Cloudinary), affichage du nom du fichier avec un bouton pour l'ouvrir dans un nouvel onglet.
 - **Section Notes** : Un grand bloc de texte libre pour écrire ses arguments de vente ou les questions posées par le recruteur.
 - **Zone Danger** : (Delete) tout en bas pour supprimer l'offre du tableau de bord et de la BDD, via un bouton "Supprimer la candidature".

9. CI/CD et Architecture cloud

L'application repose sur une architecture de déploiement continu (CI/CD) moderne et un environnement de développement conteneurisé. Le cycle de vie du code, du développement local jusqu'à la production, est automatisé et standardisé.

9.1. Environnement Local (Docker)

Afin de garantir une portabilité absolue d'une machine à l'autre, le développement local ne s'appuie pas sur des installations natives.

- **Le principe** : Le projet utilise un `.devcontainer`. Dès l'ouverture du projet dans l'éditeur (VS Code), Docker télécharge une image Linux standardisée, installe Node.js (v24.18.0) et configure les extensions de l'éditeur (ESLint, Prettier).
- **L'avantage** : Aucun conflit de version. L'environnement de développement est strictement identique pour tous les contributeurs, garantissant que le code "fonctionne partout".

9.2. Dépôt centralisé (Github)

L'intégralité du code source est hébergée sur un seul et même dépôt GitHub "history.job".

- Le dossier `/frontend` contient l'application React.
- Le dossier `/backend` contient l'API Express.
- GitHub agit comme le chef d'orchestre : il stocke l'historique (Git) et alerte les hébergeurs lorsqu'une nouvelle modification est poussée sur la branche principale (`main`).

9.3. Déploiement continu (CI/CD avec Netlify & Render)

Les hébergeurs sont connectés directement au dépôt GitHub pour automatiser les mises en ligne sans aucune action manuelle.

- **Le Front-end (Netlify)** : Écoute uniquement le dossier `/frontend`. Lorsqu'un changement est détecté, Netlify lance la commande `vite build`, génère les fichiers statiques optimisés, et les distribue sur son réseau mondial (CDN). Les modifications du back-end sont ignorées via un script `git diff` pour économiser les temps de compilation.
- **Le Back-end (Render)** : Écoute uniquement le dossier `/backend`. Lorsqu'un changement est détecté, Render télécharge le nouveau code, installe les dépendances (npm) et redémarre le serveur Node.js automatiquement.

9.4. La base de données Cloud (MongoDB Atlas)

La base de données n'est stockée ni sur Docker, ni sur Render. Elle est hébergée indépendamment sur le cloud MongoDB Atlas.

- **Connexion** : Que le serveur Node.js tourne en local (dans Docker) ou en production (sur Render), il communique avec la base de données via une URL de connexion sécurisée (URI) stockée dans les variables d'environnement (`.env`).
- **Avantage** : Les données sont centralisées, sécurisées et ne sont pas effacées lors des redéploiements de l'API sur Render.

10. Palette graphique



```
colors: {  
  'text': '#ffffff',  
  'background': '#070d18',  
  'primary': '#3067fd',  
  'secondary': '#a3b9ff',  
  'accent': '#e496a9',  
},
```

Annexe — Glossaire

Axios : Bibliothèque Front-end qui gère l'envoi des requêtes HTTP (GET, POST...) depuis React vers l'API Node.js pour récupérer ou envoyer des données.

JWT - JSON Web Token : Standard de sécurité agissant comme un badge numérique crypté. Généré à la connexion, il prouve l'identité de l'utilisateur à chaque action sans nécessiter de re-connexion.

Multer : Outil côté Back-end (Node.js) qui intercepte et décode les fichiers physiques (comme les PDF des CV) envoyés depuis un formulaire pour les rendre exploitables.

CORS : Règle de sécurité côté Back-end qui autorise spécifiquement votre site Front-end (Netlify) à interroger votre API (Render), bloquant ainsi les requêtes de sites non autorisés.

Dotenv : Bibliothèque Back-end permettant de cacher les informations sensibles (mots de passe, clés d'API) dans un fichier .env local, évitant leur exposition publique sur GitHub.

Vite.js : Outil de développement ultra-rapide pour React. Il sert de serveur local pour coder en temps réel et compile le projet en fichiers optimisés pour la production.

Netlify : Plateforme qui héberge et distribue uniquement la partie visuelle de votre site (votre code React compilé) de manière très rapide à travers le monde.

Render : Le serveur permanent qui fait tourner votre code Node.js, gère la logique complexe, vérifie les mots de passe et communique avec la base de données.

Zod : Bibliothèque de validation (Front et Back) qui garantit que les données entrantes respectent un format strict (ex: email valide) avant de les traiter.

TanStack Query : Outil Front-end qui gère automatiquement la mise en cache, la synchronisation et le rafraîchissement des données venant du serveur, remplaçant la gestion manuelle des chargements.